```
    <script src="hello.js"></script>
</head>
<body>
</body>
</html>

hello.js
window.addEventListener("load", function () {
    document.write("Hello, world again!");
});
```

Here the script has been written in a separate file called "hello.js" and has been included from "hello.htm" via a "script" tag. As soon as the browser encounters a referenced script it immediately downloads the script and runs it. In this case the script simply adds an event handler for the "window" object's "load" event and adds a line of text to the document.

JavaScript logging and debugging

During development it is fairly common to encounter situations where we need to add diagnostic output to our programs – especially during the process of debugging. The case is no different with JavaScript. Most modern browsers these days include a "console" object as part of the JavaScript runtime that can be used to log diagnostic output to a debugger console. You typically use this object like so:

```
console.log("Diagnostic output here.");
```

Where does this output show up? In the debugger's output console. In most modern browsers today you can hit the F12 key on the keyboard to launch the developer tools included in the browser. Hitting F12 on Internet Explorer 9 to launch the debugger tool window.

JavaScript Object Notation

The last topic we'll examine in this section is JavaScript Object Notation (JSON). JSON is a simple human readable textual data interchange format used by web applications primarily when interfacing with AJAX based APIs. JSON is directly supported as part of the ES5 specification and again, all modern browsers support JSON serialization and deserialization out of the box. The terse hierarchical nature of the JSON syntax lends itself well to many scenarios that had in the past been the domain of XML. If you wanted to define a "person" object for instance with a few properties, here's how you would express it using JSON syntax: